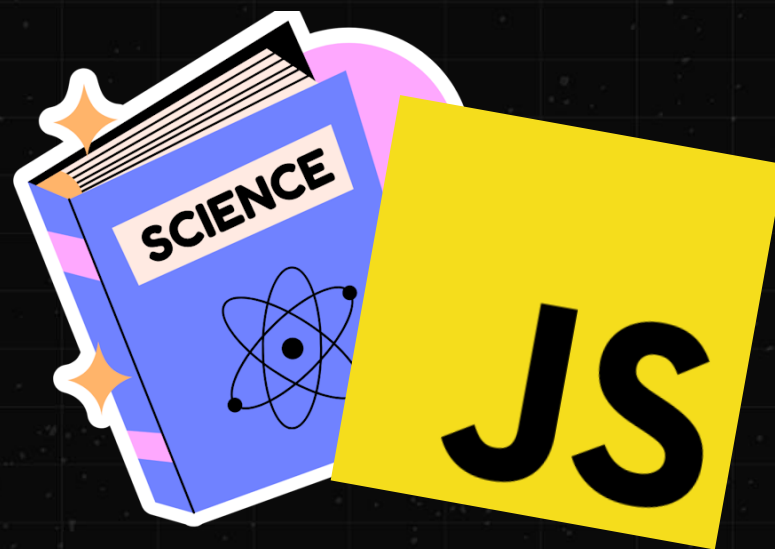


ЛЕКЦИЯ #11

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Поговорим о **представлении программы в памяти**
- * Обсудим **важнейшие сегменты** памяти и сравним их
- * Узнаем о **роли аллокатора памяти** и о том, почему это всё равно важно для frontend-разработчика

А ИЗ ЧЕГО
СОСТОИТ
ПРОГРАММА?



ДАВАЙТЕ ПЕРЕЧИСЛИМ

- * Код (функции, процедуры, классы, методы)
- * **Локальные данные** (аргументы и локальные переменные, временные объекты)
- * **Глобальные данные** (глобальные переменные и константы)
- * **Статические данные** (литералы, временные объекты)
- * **Внешние динамические библиотеки**
(разделяемые библиотеки, плагины, нативные модули)

НАЧНЕМ С ФУНКЦИЙ

- * Подпрограмма, набор инструкций
- * Может иметь входные аргументы
- * Содержит локальные переменные и возвращает результат
- * Может запускать внутри себя другие функции (в том числе себя – рекурсия)
- * После завершения локальные данные должны освободить занимаемую память



```
// Аргументы a и b - это локальные данные
function sum(a, b) {
  return a + b; // Возвращаемое значение
}

// Аргументы a и b - это локальные данные
function multiply(x, y) {
  // Тоже локальные данные
  let result = 0;

  for (let i = 1; i <= y; i++) {
    result = sum(result, x); // Вложенный вызов функции
  }

  return result; // Возвращаемое значение
}

// Получили результат, а вся локальная память освобождается
console.log(multiply(2, 3));
```

А КАК **ОЧИЩАТЬ**
ПАМЯТЬ ПОСЛЕ
ЗАВЕРШЕНИЯ
ФУНКЦИИ?



МОЖНО ПРИДУМАТЬ РАЗНЫЕ СТРАТЕГИИ

- * Функции являются, вероятно, самой нагруженной частью программы, значит их работа должна быть максимально оптимизирована
- * Функция не может выполняться «чуть-чуть» — она **всегда выполняется целиком**
- * При этом функции исполняются в порядке **«последним пришёл — первым ушёл»**
- * Например, если внутри функции вызывается другая функция, то выполнение внешней **приостанавливается и запускается внутренняя**

МЫ ПОМНИМ, ЧТО ПАМЯТЬ – ЭТО МАССИВ

- * Мы можем работать с ней **как с массивом поменьше**
- * Или **формировать другие структуры** данных
- * Чем проще и ближе наша структура к структуре массива, тем она **эффективнее и проще в реализации**
- * Можно понять, что лучшей структурой для локальной памяти функций будет **стек, реализованный на обычном массиве**

ПОЧЕМУ ИМЕННО СТЕК

- * Когда **функция вызывается**, в стек помещается (push) её память: аргументы, локальные переменные и адрес возврата
- * Когда функция завершается, её память **извлекается из стека (pop)**
- * Если функция А вызывает функцию В, **стек растёт: после памяти А кладётся В**
- * При **возврате из В её память удаляется**, возвращая управление А
- * Благодаря этому последний вызванная функция возвращается первой — **принцип LIFO** (Last In, First Out)

```
1 function main() {  
2   const x = 10;  
3   const y = 20;  
4   ◀ const result = calculate(x, y);  
5   console.log(result);  
6 }  
7  
8 function calculate(a, b) {  
9   const sum = a + b;  
10  ◀ const isValid = validate(sum);  
11   return isValid ? sum : 0;  
12 }  
13  
14 function validate(value) {  
15   return value > 0;  
16 }  
17
```

▼ Top (Last In) ▼

TOP

main()

x : 10
y : 20
result : ??? (ожидает calculate)
return addr: global

calculate(a = 10, b = 20)

a : 10
b : 20
sum : 30
isValid : ??? (ожидает validate)

✓ validate(value = 30)

value : 30
return addr: calculate line 11
→ возвращает boolean (true/false)

△ Bottom (First In) △

✚ Принцип LIFO (Last In — First Out)

- **validate()** вызвана последней → её кадр находится на вершине стека (Top)
- При завершении **validate()** её кадр удаляется первым → управление возвращается в **calculate()**
- Затем завершится **calculate()** → возврат в **main()** → потом завершение программы

Порядок: main() [PUSH] → calculate() [PUSH] → validate() [PUSH] → validate() [POP] → calculate() [POP] → main() [POP]

PUSH

→

PUSH

→

PUSH

→

POP

→

POP

→

POP



main()



calculate()



validate() — активный в данный момент



текущий вызов (top of stack) --- связь вызова



Рекурсия без базового случая может привести к переполнению стека (Stack Overflow)

НО ВЕДЬ
У ТАКОГО СТЕКА
ФИКСИРОВАННЫЙ
РАЗМЕР?



РАЗУМЕЕТСЯ

- * Поэтому выделим память **под стек** достаточного размера, например, 8 мегабайт
- * При достижении конца стека **выделим исключительную ситуацию** – переполнение стека (или обозначим это как UB)
- * Каждый **вызов функции будет иметь свою память на стеке**, именуемую кадром
- * Для добавления новых значений **нужно знать размер добавляемых данных**
- * Когда функция завершается, она должна **передвинуть вершину стека** в то место, где она была до вызова

СЕГМЕНТ СТЕКА ПОДДЕРЖИВАЕТСЯ АППАРАТНО

- * Процессор имеет два специальных регистра для работы с памятью как со стеком
- * Регистр RBP содержит **адрес начала актуального кадра стека** (граница кадра)
- * Регистр RSP **содержит адрес актуальной вершины стека**
- * Благодаря этим регистрам, работа с сегментом стека **становится ещё эффективнее**
- * Но сама реализация функций остаётся **задачей компилятора или среды исполнения**
- * **В языках ассемблера функций нет**, поэтому сам разработчик решает как реализовать нужное ему поведение

А КАКОЙ
В **ДЕЙСТВИТЕЛЬНОСТИ**
РАЗМЕР СТЕКА?



ЗАВИСИТ ОТ РАЗНЫХ ПАРАМЕТРОВ

- * Различные ОС имеют **разные значения размера стека по умолчанию**
- * В Linux 8 МБ для основного потока, а в Windows 1 МБ
- * Можно **ограничивать размер стека** с помощью параметров компиляции
- * В JS размером стека управляет **виртуальная машина**
- * В Node.js можно управлять флагом `--stack-size`

НАПРИМЕР



Увеличить размер стека до 2 МБ (2048 КБ)

```
node --stack-size=2048 app.js
```

Или для отладки рекурсивной функции

```
node --stack-size=4096 --inspect recursive.js
```

КАК ФУНКЦИЯ
ПОНИМАЕТ,
КУДА **ВЕРНУТЬСЯ**
ПОСЛЕ СВОЕГО
ЗАВЕРШЕНИЯ?

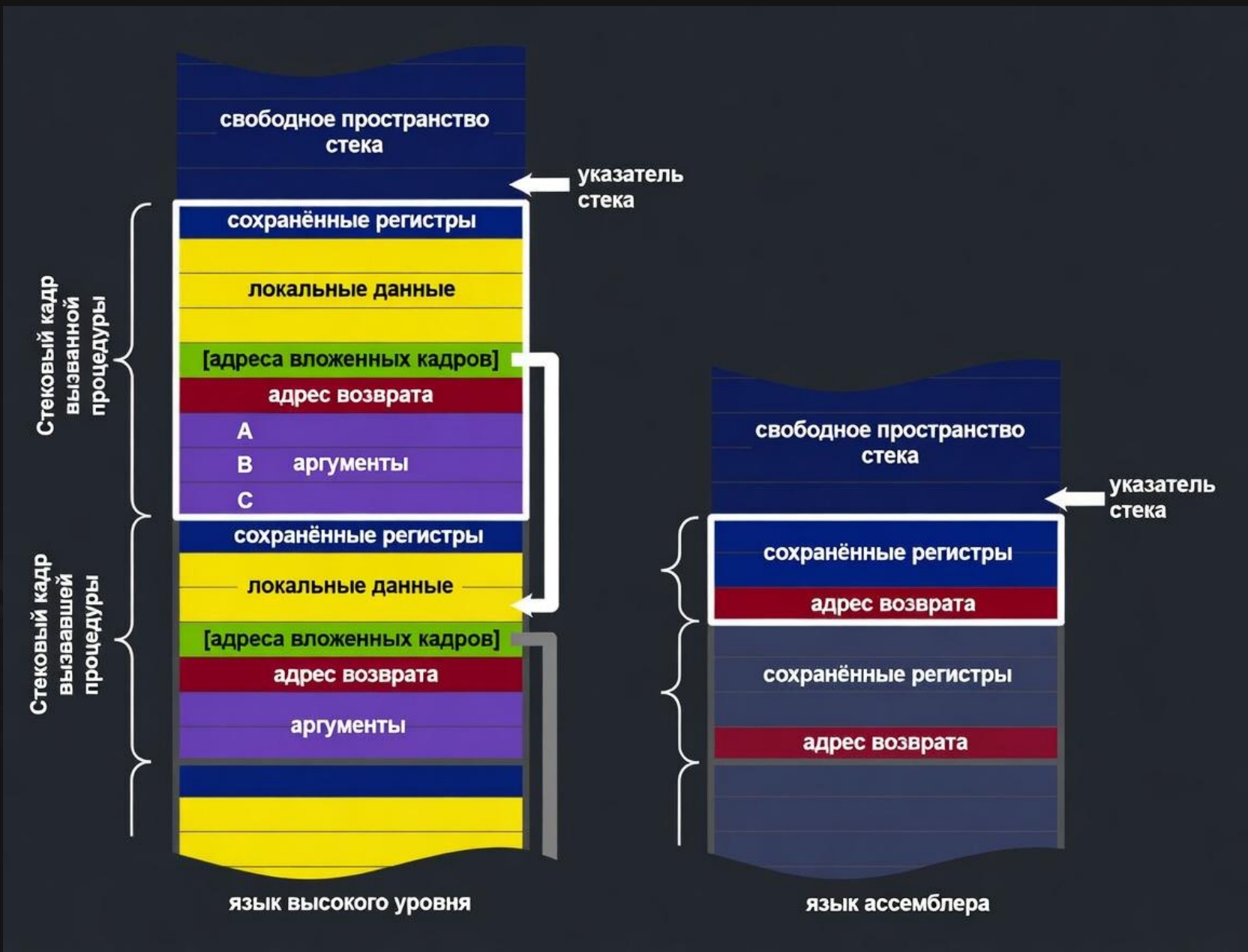


ОНА СОХРАНЯЕТ ЭТО НА СТЕКЕ

- * Когда мы вызываем функцию, она первым делом **запоминает адрес возврата** на стеке (место, где вызвали функцию)
- * А также **сохраняет значение RBP** родительской функции
- * После этого **выделяется фиксированная память** под локальные переменные вызываемой функции и устанавливаются новые значения RBP и RSP
- * Эти **необходимые для работы приготовления** называются прологом функции
- * А **при её завершении** выполняется эпилог

ЭПИЛОГ

- * Эпилог выполняется при завершении функции
- * **Переместить вершину стека (RSP) на начало текущего кадра**
[тем самым освободив память локальных переменных]
- * **Восстановить сохранённое значение RBP** родительской функции
[указатель начала её кадра]
- * **Перейти по адресу возврата** обратно в вызывающую функцию



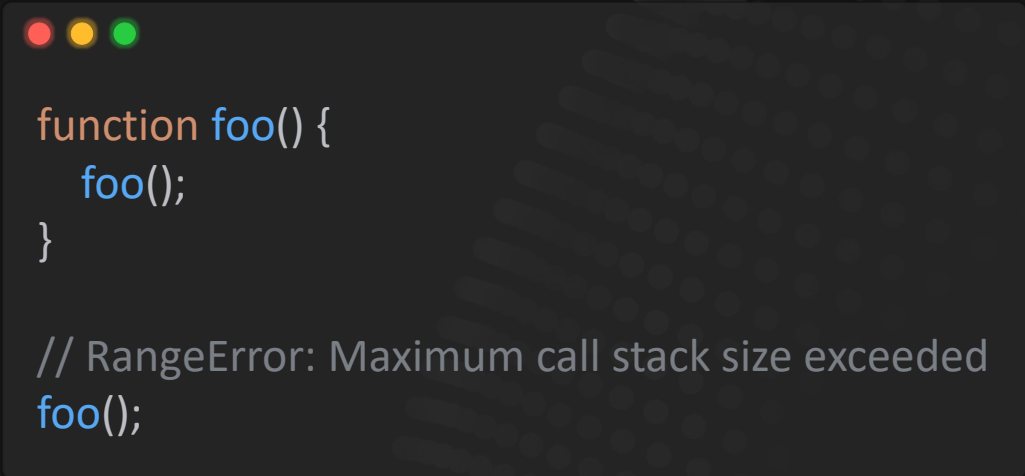
ПОЛУЧАЕТСЯ, ЧТО ФУНКЦИЯ
БЕЗ ЛОКАЛЬНЫХ ДАННЫХ
ВСЕ РАВНО ЗАНИМАЕТ СТЕК?



ДА

- * Да, потому что в кадре всё равно сохраняются адрес возврата и старый RBP
- * Поэтому выполнение такой функции в бесконечной рекурсии приведёт к ошибке переполнения стека
- * Однако компилятор или среда выполнения могут оптимизировать некоторые вызовы функций, чтобы они не создавали отдельные кадры стека
- * Это называется **хвостовой оптимизацией** (tail call optimization)

НАПРИМЕР



```
function foo() {  
  foo();  
}  
  
// RangeError: Maximum call stack size exceeded  
foo();
```

КАК ФУНКЦИЯ ВОЗВРАЩАЕТ
ДАННЫЕ, ЕСЛИ ЕЁ КАДР
ОСВОБОЖДАЕТСЯ?



ПРИ ВОЗВРАТЕ ПАМЯТЬ ВСЕГДА КОПИРУЕТСЯ

- * Возвращение указателя на данные в стеке — это гарантированный способ получить UB
- * **Безопасные языки запрещают такое делать**
- * Обычная стратегия — **полное копирование данных на стеке** как при вызове функции, так и при завершении
- * Для избежания избыточных копирований многие значения хранятся в других сегментах памяти, а **на стеке хранится указатель на них**

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОСТАРАЙТЕСЬ ПОНЯТЬ СМЫСЛ

- * Функцию можно вызвать и получить результат — при этом мы **не думаем про локальную память функций**
- * Она выделяется и освобождается автоматически без каких-либо сборщиков мусора
- * В высокоуровневых ЯП аргументы для вызываемой функции **просто копируются при передаче**
- * Если значение хранится по **ссылке**, то копируется только она
- * Возвращаемое значение **тоже копируется**

ПО ССЫЛКЕ ИЛИ ПО ЗНАЧЕНИЮ

- * Известное утверждение в JS — что объекты передаются по ссылке, а примитивы по значению — **в корне неверно**
- * Из-за динамической типизации все значения представлены как ссылки — **благодаря этому можно статически вычислить размер кадра функции**
- * На **стеке функций в JS** находятся **только ссылки**, которые копируются (передаются по значению) при передаче
- * Однако некоторые **ссылки таковыми не являются** и хранят значения в себе (SMI, логические значения и другие)

А ГДЕ **ХРАНИТСЯ** ТО,
ЧТО НЕ НА СТЕКЕ?



У СТЕКА ЕСТЬ ДВА ВАЖНЫХ ОГРАНИЧЕНИЯ

- * Обязательно копирование при возврате значения из функции
- * Нужно **заранее** знать **размер**, который будет занимать значение в памяти
- * Поэтому тот же вектор на стеке сделать не получится
- * Для обхода этих ограничений придумали **другой сегмент памяти** – кучу (heap)

КОНТРАКТЫ КУЧИ

- * При работе с кучей у нас возникает три естественных запроса
- * **Выделить память заданного размера и вернуть указатель на первый байт (alloc)**
- * **Перевыделить память нового размера для уже имеющегося блока и вернуть указатель на первый байт (realloc)**
- * **Освободить память по заданному указателю (free)**

СЛОЖНОСТИ РЕАЛИЗАЦИИ

- * **Сегмент стека сделать очень просто** — выделить память фиксированного размера и два специальных регистра для управления
- * **С кучей такое сделать куда сложнее**, поэтому на аппаратном уровне есть только простейшие регистры для доступа к любой ячейке памяти по её адресу
- * А сложный функционал — дефрагментация памяти и быстрый поиск — нужно **запрограммировать самостоятельно, используя более сложные структуры**
- * Такая программа называется **аллокатором**, и она является важной частью ОС
- * Но есть задачи, где нужно использовать **альтернативный аллокатор**

ЧТО **КОНКРЕТНО**
НАМ ДАЕТ КУЧА?



ДИНАМИЧЕСКАЯ ПАМЯТЬ

- * Мы можем «просить» у аллокатора память произвольного размера для наших данных и освободить её, когда она уже не нужна
- * Эти данные больше не привязаны к времени жизни функции
- * На стеке просто хранится указатель на эти данные
- * Все объекты JS и некоторые примитивы хранятся исключительно в куче
- * Данные в замыкании также работают через кучу

ПРОБЛЕМЫ С КУЧЕЙ

- * Потребность в **явном выделении и освобождении памяти** имеет последствия
- * В низкоуровневых языках легко забыть освободить память или освободить её дважды — это приводит к UB
- * В высокоуровневых языках за освобождение отвечает сборщик мусора (GC) — **подсистема рантайма, которая автоматически освобождает неиспользуемую память**
- * Альтернативный подход — **счётчики ссылок** — память освобождается сразу, как только счётчик достигает нуля
- * Или управление памятью может быть сделано на этапе компиляции (Rust)

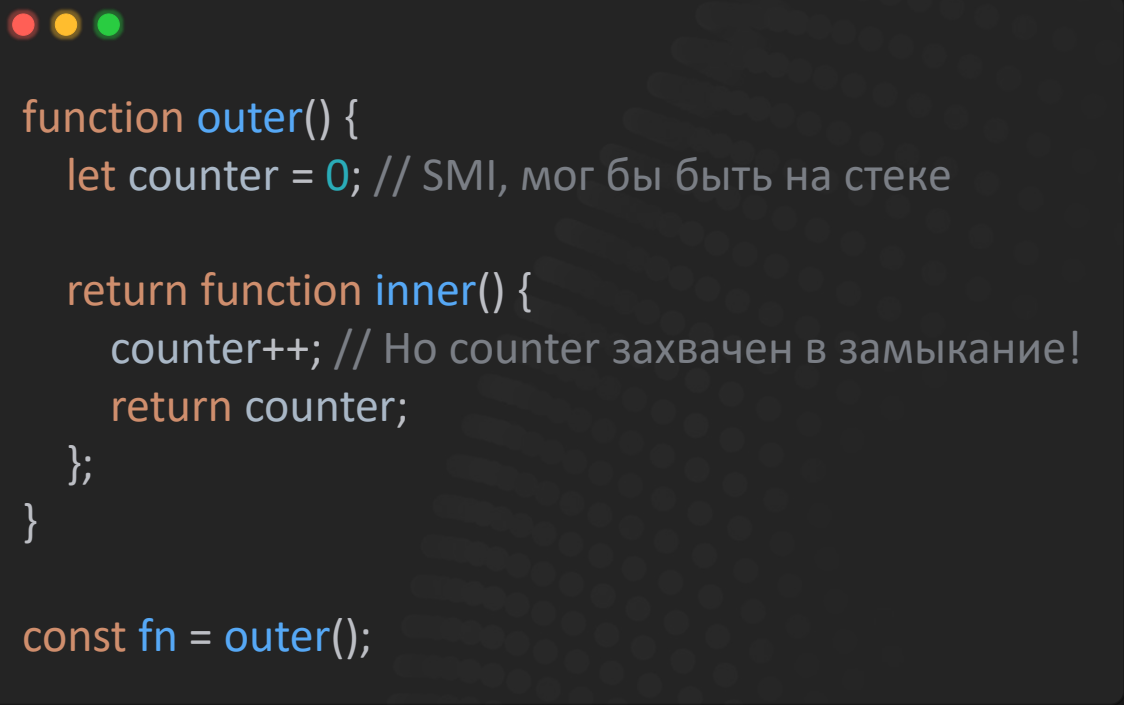
СТЕК И КУЧА

- * Часто можно слышать тезис, что **стек быстрее кучи**, поэтому языки типа Rust или C/C++ стараются его активно использовать
- * Это правда, **но с некоторыми уточнениями**
- * Стек очень прост в реализации и **имеет константную сложность**
- * Операции **выделения и освобождения памяти** в куче куда дороже
- * Функциям все равно нужен стек, а потребность ходить лишний раз по памяти **ухудшает локальность и эффективность кэша**
- * Но стек **проигрывает при копирование больших данных**

ЧТО КУДА ПОЛОЖИТЬ

- * В JS мы **не можем явно управлять** тем, что лежит на стеке, а что в куче
- * Однако мы можем косвенно понять по типу данных, где они будут располагаться
- * SMI-числа, логические значения и другие константы скорее всего будут расположены на стеке
- * Однако замыкание может изменить это поведение

НАПРИМЕР



```
function outer() {  
  let counter = 0; // SMI, мог бы быть на стеке  
  
  return function inner() {  
    counter++; // Но counter захвачен в замыкание!  
    return counter;  
  };  
}  
  
const fn = outer();
```

МАССИВ SMI ХРАНИТСЯ НА СТЕКЕ?



НЕТ

- * Массив — это объект, а значит, **должен располагаться в куче**
- * Если его элементы — это SMI, то они **всё равно хранятся в куче** (но без лишних ссылок)
- * К тому же **массивы в JS могут расширяться** — стек не подходит в любом случае
- * Однако VM может **оптимизировать простые литеральные формы** массивов внутри функции

НАПРИМЕР

```
function fast() {  
  let arr = [1, 2, 3]; // Маленький массив-литерал  
  
  // VM может выделить его на стеке (оптимизация)  
  return arr[0] + arr[1] + arr[2];  
}
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Главное поймите – **идеальной структуры данных нет**
- * стек отлично подходит для локальных данных функции, **но он имеет и ограничения**
- * Куча решает эти проблемы, но **ценою усложнения и понижением эффективности**
- * Есть ЯП, которые в принципе **не используют кучу, а только стек**
- * Например, старые версии Fortran

СТЕК, КУЧА И WASM

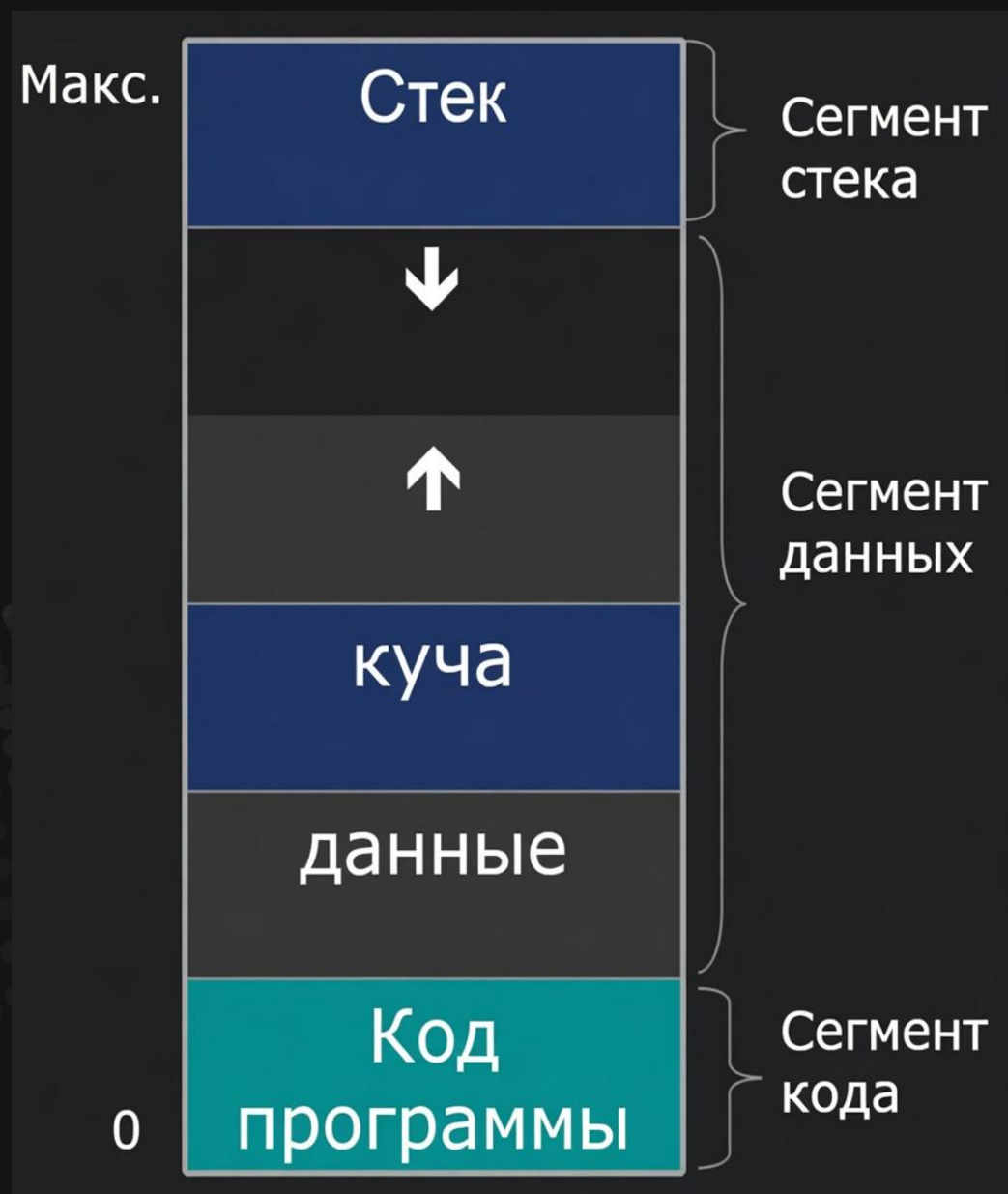
- * Есть конкретная причина, зачем вам нужно глубоко понимать устройства этих сегментов для задач Frontend
- * Дело в том, что Wasm контейнеры представляют свою динамическую память, как **простые линейные буферы** (линейная память)
- * Для использования структур с динамической памятью вам придется **подключить или написать специальный аллокатор**
- * А для оптимизаций явно оперировать стеком

А ГДЕ КУЧА
ХРАНИТСЯ
В ПАМЯТИ?



ЕЁ РАЗМЕЩАЮТ НА ДРУГОМ КОНЦЕ К СТЕКУ

- * Это позволяет стеку и куче **«расти»** навстречу друг другу
- * Если они **встретятся**, то это будет UB
- * В отличие от стека, **размер кучи может меняться динамически**
- * Но в JS он **ограничен параметрами виртуальной машины**
- * Например, в Chrome это 2 гигабайта
- * В Node.js **можно явно управлять размером кучи**
с помощью специальных флагов



```
import v8 from "node:v8";

const { heap_size_limit } = v8.getHeapStatistics();
const heapSizeInGB = heap_size_limit / (1024 * 1024 * 1024);

// {
//   rss: 52871168,
//   heapTotal: 5902336,
//   heapUsed: 4534168,
//   external: 1702514,
//   arrayBuffers: 22975
// }
console.log(process.memoryUsage());

// 4.1875 GB
console.log(`${heapSizeInGB} GB`);
```

```
node --max-old-space-size=4096 test.js
```

В V8 ПРИМЕНЯЕТСЯ ДВЕ СТРАТЕГИИ

- * **Молодое поколение (New Space)** — объекты, которые «быстро умирают»; живут в небольшом сегменте памяти (по умолчанию ~16–32 МБ)
- * **Старое поколение (Old Space)** — объекты, которые «живут долго»; перемещаются в большой сегмент кучи
- * Это важно для эффективной сборки мусора и это можно тюнить
- * [understanding-and-tuning-memory](#)

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

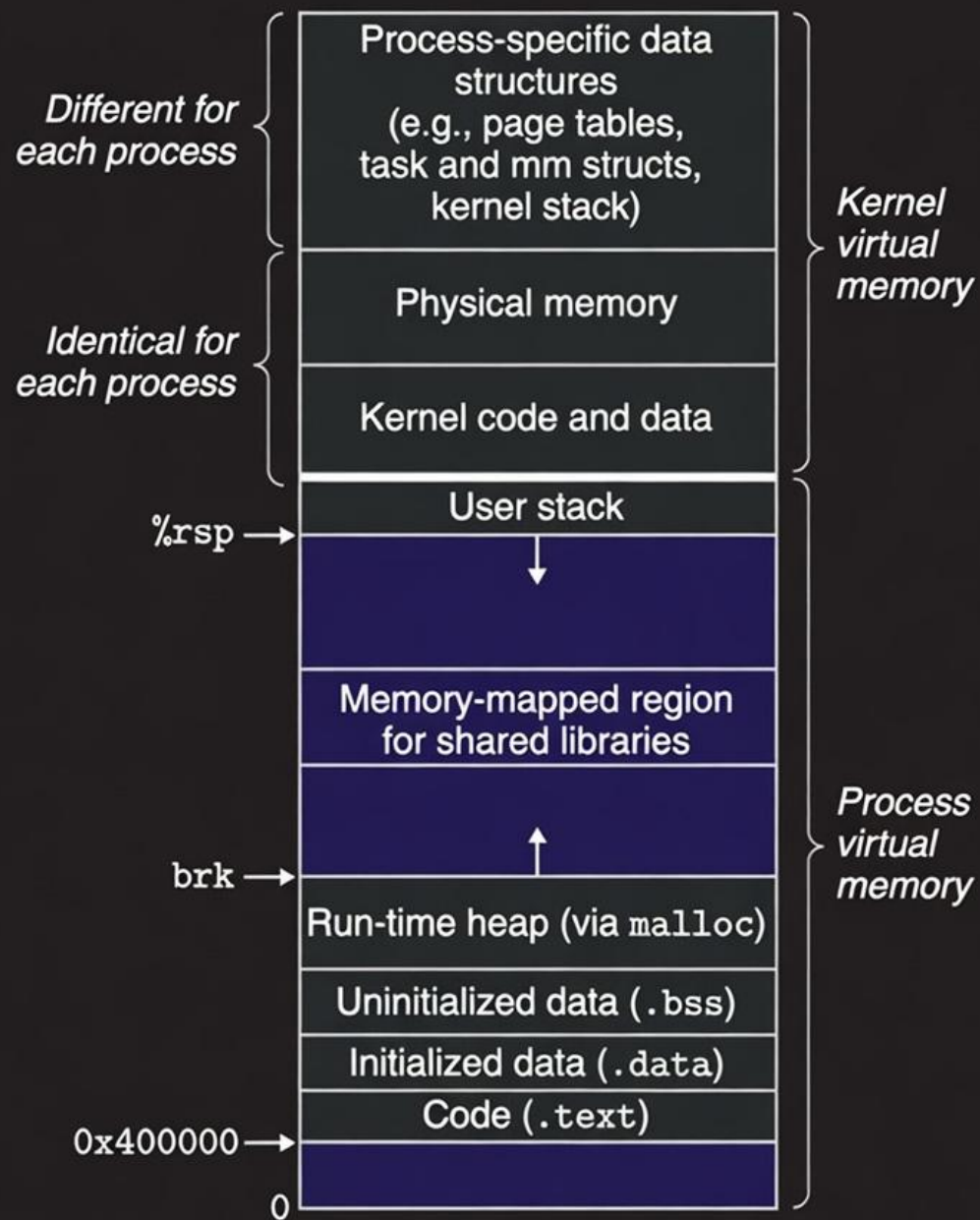
- ✱ Мы разобрали **два важнейших сегмента памяти программы**
- ✱ Но кроме стека и кучи есть и другие сегменты

В ПРОГРАММЕ МОГУТ БЫТЬ ДАННЫЕ

- * Имеющие глобальную область видимости
- * Размер которых известен
- * Могут быть неизменяемыми
- * Например, литералы строк
- * Держать их в куче или стеке не эффективно
- * Нужна еще область памяти...

Figure 9.26

The virtual memory of a Linux process.



ЧТО ЕЩЕ ВАЖНО

- * Разные сегменты **могут иметь разные права**
- * стек и кучу **можно менять и читать**
- * Область с кодом **можно исполнять и читать, но нельзя менять**
- * И так далее

АНАТОМИЯ ПРОГРАММЫ НА JS ТАКАЯ ЖЕ?



ТУТ ЕСТЬ НЮАНС

- * Программы на JS **не исполняются процессором напрямую**
- * Вместо этого они исполняются другой программой – **виртуальной машиной (VM)**
- * А вот VM уже **может*** исполняться напрямую процессором
- * Например, **V8** написан на C++ **и компилируется в машинные коды** под конкретную архитектуру
- * **Анатомия памяти V8** соответствует тому, что мы обсудили (стек, куча, сегменты)
- * V8 использует различные сегменты памяти для хранения значений JS

ВАЖНО ПОНЯТЬ ОДНУ ВЕЩЬ

- * В базовом представлении **вся память — это просто массив** (байт или слов)
- * **Сегмент** определяет **контракт** — как именно мы работаем с этой памятью (чтение, запись, исполнение, рост и другое)
- * Для эффективной работы процессор может предоставлять **специальные регистры** и **инструкции** (например, SS, SP для стека)
- * Какие сегменты и как работают с памятью — определяют **реализация** и **дизайн самого языка программирования**

НАПРИМЕР

- ✱ В языках Algol-60 или Fortran вплоть до версии 90 не использовалась куча
- ✱ Все данные размещались исключительно на стеке
- ✱ Несмотря на явные ограничения **у такого подхода есть и свои плюсы**

А КАК ЖИТЬ
БЕЗ КУЧИ?



ВСЕ ОПРЕДЕЛЯЕТСЯ ДИЗАЙНОМ

- * В нашем ЯП может просто **не быть массивов динамической длины** и основанных на них структур
- * Или в нашем ЯП в принципе все данные неизменяемые
- * Тогда мы могли бы реализовать всё без использования кучи вообще
- * Либо использовать её очень ограниченно

**A KAK CO
CTEKOM?**



СМОТРИТЕ

- * Процессор умеет делать **вычисления только над данными в регистрах**
- * Чтобы данные оказались в регистре, их нужно **считать из памяти** (или разместить там явно)
- * **Доступ к памяти** может быть долгим *
- * Пока **процессор ждёт данные**, он простаивает
- * Эта **проблема частично решается**: кэшем и концепцией «виртуальных ядер»
- * Но если данные уже в регистрах — такой проблемы просто нет

НО ТУТ ЕСТЬ НЮАНСЫ

- * Операции над регистрами **возможны в очень низкоуровневых ЯП**, например, в языках ассемблера
- * В остальных случаях реализация может использовать регистры **при генерации машинного кода или исполнении**
- * Это скорее **оптимизация**, но очень существенная

ЗАЧЕМ НАМ СТЕК?

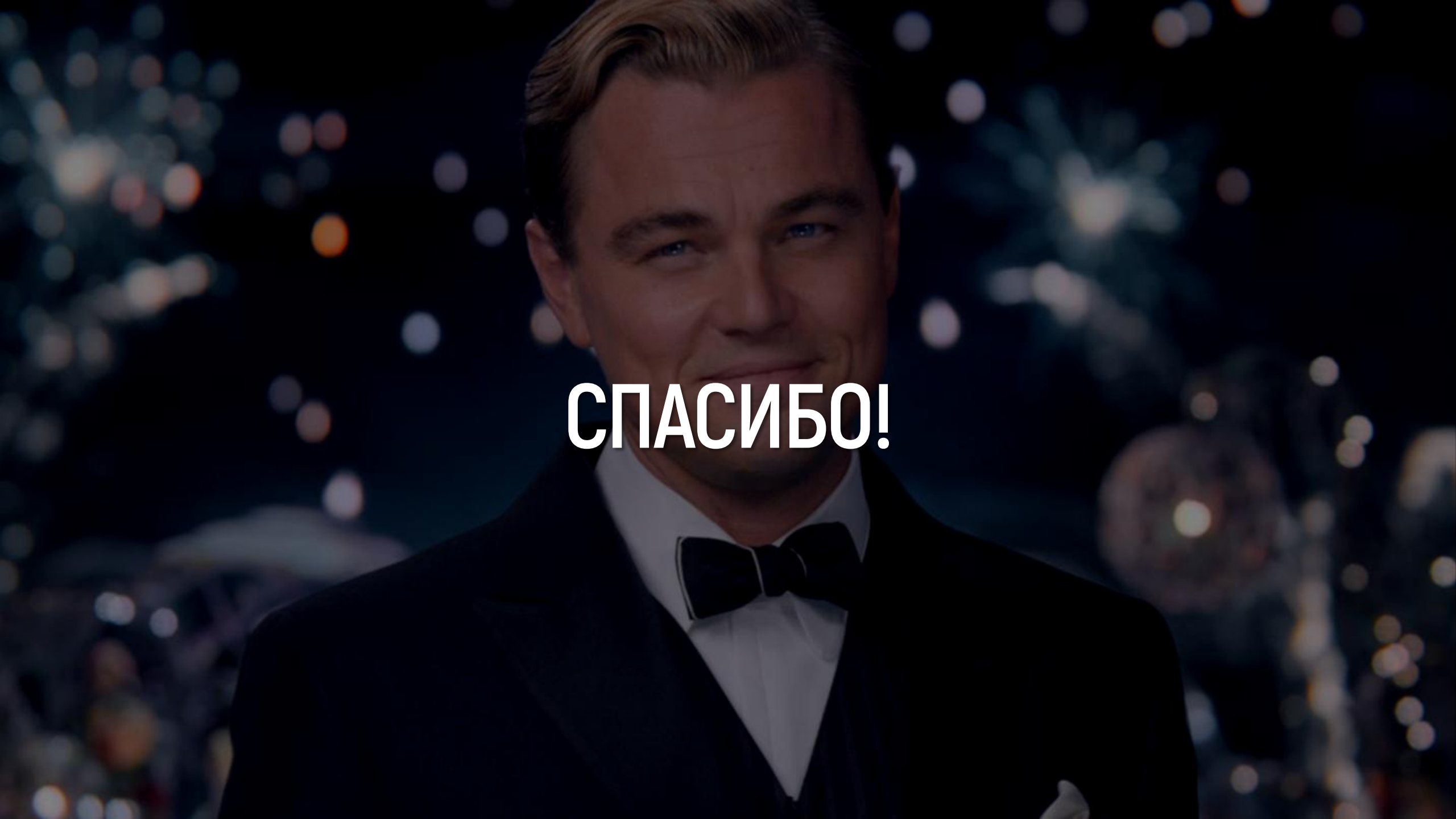
- * Для того чтобы работали функции
- * Но мы можем **избавиться от функций в момент компиляции** (инлайнинг)
- * Или мы можем не использовать стек для аргументов, возвращаемых значений, а **сразу использовать регистры**
- * **Существуют специальные соглашения** какие регистры использовать в таких кейсах
- * Реализации использующие такие подходы могут быть очень эффективны

НО ЕСТЬ И ОГРАНИЧЕНИЯ

- * Регистров мало и размер их невелик
- * Набор регистров **определяется конкретным процессором и его архитектурой**
- * Усложнение реализации

ПОДВЕДЕМ ИТОГИ

- * **Стек и куча — два важнейших сегмента памяти**
- * **Стек нужен функциям** — они размещают в нём свои локальные данные
- * **Стек привязан ко времени жизни функции**
и автоматически очищается при её завершении
- * **Куча не привязана ко времени жизни функции и может динамически расширяться**
- * **Благодаря куче можно создавать структуры с переменным размером**
- * **Например, вектор или хэш-таблица**
- * **Кроме кучи и стека, существуют и другие сегменты памяти**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with numerous out-of-focus, colorful bokeh lights in shades of blue, white, and orange, suggesting a night-time event or fireworks.

СПАСИБО!